

Capítulo 3 — GUÍA DE LECTURA

Capítulo 3 — GUÍA DE LECTURA.....	1
JavaScript: el lenguaje del navegador y su bucle de eventos	3
El único de los tres lenguajes que ejecuta instrucciones, explicado en criollo	3
Antes de empezar: cómo usar esta guía.....	3
3.1 — De qué se trata esta clase	3
Qué dice	3
En criollo.....	4
3.2 — Por qué JavaScript es como es: diez días de 1995	4
Qué dice	4
En criollo.....	4
Tres linajes que nunca habían convivido	5
La cronología, que también fue marketing	5
Las cuatro decisiones que explican todo lo demás	6
3.3 — Cómo y cuándo se ejecuta tu código.....	7
Qué dice	7
En criollo.....	7
3.4 — Tipos: los primitivos y todo lo demás	8
3.4.1 — Los siete primitivos	8
Qué dice	8
En criollo.....	8
3.4.2 — Valor y referencia: dos nombres para el mismo objeto	8
Qué dice	8
En criollo.....	9
3.4.3 — El envoltorio que aparece y se va	9
3.5 — Coerción: cuando el lenguaje convierte por vos	10
3.5.1 — Los ocho valores falsos	10
Qué dice	10
En criollo.....	10
3.5.2 — Las dos igualdades, y cuál usar siempre	10
3.5.3 — NaN y los valores que no están.....	11
3.5.4 — El modelo numérico y el problema del dinero	11
Qué dice	11
En criollo.....	11
3.6 — Declaraciones, ámbito y clausuras	12
3.6.1 — var, let y const	12
3.6.2 — Elevación y la zona muerta temporal	12
3.6.3 — Clausuras: la función que se acuerda	13

Qué dice	13
En criollo.....	13
3.7 — Funciones, y el <code>this</code> que depende de cómo llamas	14
3.7.1 — Tres formas de escribir una función	14
3.7.2 — <code>this</code>	14
Qué dice	14
En criollo.....	14
3.8 — Objetos, prototipos y clases.....	15
Tres comportamientos donde se ve que no es una clase de verdad	16
3.9 — Módulos.....	17
Las cuatro propiedades que hay que tener presentes	17
Dos formas de exportar, y la elección no es indistinta	17
3.10 — El bucle de eventos.....	18
3.10.1 — Un solo hilo.....	18
Qué dice	18
En criollo.....	18
3.10.2 — Pila, tareas y microtareas	18
Qué dice	18
En criollo.....	20
3.10.3 — El mismo problema del otro lado del cable	21
3.11 — Herramientas de diagnóstico	21
Las herramientas, y cuándo usar cada una	21
3.12 — Seguridad y evolución.....	23
Tres riesgos concretos	23
Y de acá sale TypeScript, que es el próximo problema.....	23
3.13 — Verificación: el checklist honesto	24
3.14 — Los diez errores frecuentes.....	24
3.15 — Las actividades, y qué busca cada una	25
1. Predicción del bucle de eventos	25
2. Referencias en el dominio del TPI.....	25
3. Auditoría de coerción.....	25
4. El problema del dinero.....	25
5. Estado privado con clausuras	25
6. Exploración: el costo de bloquear	26
7. Exploración: la compatibilidad hacia atrás.....	26
3.16 — Síntesis: las once frases	26
3.17 — Qué leer, y en qué orden	26
Si lees una sola cosa.....	27

Si lees tres	27
Las fuentes normativas (para consultar, no para leer de corrido)	27
Cierre: las siete cosas que hay que recordar	27

JavaScript: el lenguaje del navegador y su bucle de eventos

El único de los tres lenguajes que ejecuta instrucciones, explicado en criollo

Traducción didáctica del texto académico, sección por sección. Mismos conceptos, otro idioma.

Antes de empezar: cómo usar esta guía

Este documento no reemplaza al capítulo original: lo traduce. El texto académico está escrito en el idioma de los papers —denso, comprimido—; esta guía lo desarma y lo cuenta como se lo contarías a alguien en un café. La regla es una sola: **no se pierde ni un concepto**. Si el original dice *coerción*, acá dice *coerción*; si nombra ECMA-262, el estándar del WHATWG, la norma IEEE 754 o la regla RN-F08, acá están las cuatro.

Cada sección tiene tres partes:

- **Qué dice** — la idea del original, en dos o tres oraciones.
- **En criollo** — la explicación larga, con la analogía que la hace pegar.
- **Para el pizarrón** — la frase que te tenés que llevar.

Y un aviso: **este es el primer capítulo del módulo donde se programa**. Vos ya sabés programar, así que la dificultad no va a estar donde la esperás.

LA IDEA MADRE DE TODO EL CAPÍTULO

Si te quedás con una sola frase de las cuarenta páginas, que sea esta:

JavaScript no se diseñó: se publicó. Y desde 1995 nadie pudo corregir nada, porque corregir algo rompería páginas que todavía funcionan.

La coerción, el `typeof null` que devuelve "object", el `var` que ignora los bloques, el `this` que cambia según cómo llames: **nada de eso es un error de diseño suelto. Es un error de diseño que ya no se pudo corregir**, con treinta años de arreglos apilados al lado.

Por eso el capítulo empieza por 1995 y no por la sintaxis: **conocer el problema que una decisión vino a resolver es la única forma de saber si podés confiar en ella.**

3.1 — De qué se trata esta clase

Qué dice

Los dos capítulos anteriores describieron una plataforma: un protocolo que trae documentos y un lenguaje declarativo que decide cómo se ven. Este introduce el tercero, el único que ejecuta instrucciones. Y como el alumno que llega ya sabe programar —variables, funciones, condicionales, bucles y estructuras de datos, de Programación 1 y 2—, **el capítulo no enseña a programar en JavaScript: enseña en qué se diferencia JavaScript de lo que ya sabe**. Las diferencias no se limitan a la sintaxis.

En criollo

Vos abris un archivo `.js`, ves llaves, ves `if`, ves `for`, y pensás: «esto ya lo sé, es Java con menos ceremonia». **Esa impresión es correcta durante unas cuarenta líneas y después te arruina la tarde**, porque las diferencias no están en la sintaxis —la parte que se ve— sino abajo, en el modelo. Tres de ellas gobiernan todo lo demás:

Si no sabés esto...	...no vas a entender esto otro
La coerción: convierte tipos en vez de fallar	Por qué un <code>if</code> con un arreglo vacío entra siempre. Es la misma decisión del HTML que nunca rechaza marcado (Capítulo 1) y del CSS que descarta en silencio lo que no entiende (Capítulo 2)
El modelo numérico: un solo tipo de número , que no representa exactamente <code>0.1</code>	Por qué el TPI exige que todo importe viaje como cadena: RN-F08 nace de un límite del punto flotante (sección 3.5.4)
El modelo de ejecución: un solo hilo , el mismo que dibuja la pantalla	Por qué una operación que tarda no da una pantalla lenta sino una congelada . La asincronía del Capítulo 5 existe para esquivar eso (sección 3.10)

Ese último punto tiene un puente con el TPI: **la sección 1.4** describe el modelo de ejecución asíncrono del backend y **la sección 5.5** exige que el hash de contraseña salga del bucle de eventos. Mismo problema, otro lado del cable (sección 3.10.3).

PARA EL PIZARRÓN

El objetivo de la clase es concreto y se puede medir: al terminar tenés que poder **agarrar un fragmento de código asíncrono y predecir en qué orden se van a ejecutar sus partes**, antes de correrlo.

No es un juego de ingenio: es la habilidad que el Capítulo 5 **da por supuesta**, y la que separa a quien depura una promesa de quien le agrega `setTimeout` hasta que anda.

Lo que vas a recorrer: origen y diseño de JavaScript; cómo y cuándo se ejecuta el código de una página; primitivos y objetos, con valor y referencia; coerción y valores falsos; el modelo numérico y sus límites; declaraciones, ámbito, elevación y zona muerta temporal; clausuras; funciones y `this`; objetos, prototipos y clases; módulos; el bucle de eventos con su pila, sus tareas y sus microtareas; diagnóstico y depuración; y seguridad y evolución del lenguaje.

3.2 — Por qué JavaScript es como es: diez días de 1995

Qué dice

En 1995 las páginas eran documentos estáticos y toda interacción exigía enviar un formulario y esperar una página nueva. Netscape quería resolver eso con lo que su documentación interna llamaba **un lenguaje de pegamento**: algo simple, para autores que no eran programadores. Brendan Eich fue contratado en abril de 1995 con ese encargo y con una restricción de marketing determinante —Netscape acababa de firmar con Sun Microsystems para incorporar Java al navegador, y el lenguaje nuevo **debía parecerse a Java**—. Eich escribió el prototipo en **diez días**.

En criollo

El dato de los diez días se repite como anécdota y la anécdota tapa lo importante. **Lo importante no es que lo hizo rápido: es lo que esa velocidad implicó**. Se publicó sin el período de revisión que

habría corregido sus decisiones apresuradas, y una vez afuera —con páginas dependiendo de él— ya no se pudo cambiar nada. Y «parecerse a Java» no era *ser* Java, sino parecerse lo suficiente para presentarlo como su compañero menor: una decisión de marketing definió la sintaxis de un lenguaje que hoy corre en miles de millones de dispositivos.

Tres linajes que nunca habían convivido

Linaje	Qué aportó	Dónde lo vas a ver
C y Java · la sintaxis	Llaves, punto y coma, <code>for</code> , <code>if</code>	En todas partes: produce la ilusión de familiaridad
Scheme · funciones de primera clase	Las funciones son valores: se pasan, se devuelven, se guardan	En las clausuras (3.6.3), los manejadores del Capítulo 4 y los stores del Capítulo 8
Self · los prototipos	No hay clases: los objetos heredan de otros objetos	En el modelo de la sección 3.8, y en por qué <code>class</code> es una fachada

De ahí sale la confusión más persistente del lenguaje: **parece Java y funciona como Scheme**. Quien lo lee esperando lo primero se equivoca en todo lo que importa.

La cronología, que también fue marketing

Internamente fue **Mocha**, se publicó como **LiveScript** en septiembre de 1995 y en diciembre pasó a **JavaScript**, al cerrarse el acuerdo con Sun. **No tiene ninguna relación técnica con Java**.

Microsoft respondió en 1996 con **JScript**, obtenido por ingeniería inversa para Internet Explorer 3. Las diferencias bastaban para que un mismo script anduviera en un navegador y no en el otro —la misma guerra que el Capítulo 2 describió del lado del CSS—, y eso llevó a Netscape a presentarlo ante ECMA International: **la primera edición de ECMA-262 es de junio de 1997**, y el nombre **ECMAScript** fue un compromiso porque «JavaScript» era marca de Sun. Después, **ECMAScript 4** quiso agregar clases, tipado estático y módulos; el comité se dividió, la propuesta se abandonó en 2008, y de esa crisis salió la decisión de **avanzar en incrementos chicos**: **ES5** en 2009, **ES2015** en 2015, y desde entonces una publicación **por año** con lo que haya madurado dentro del comité **TC39**.

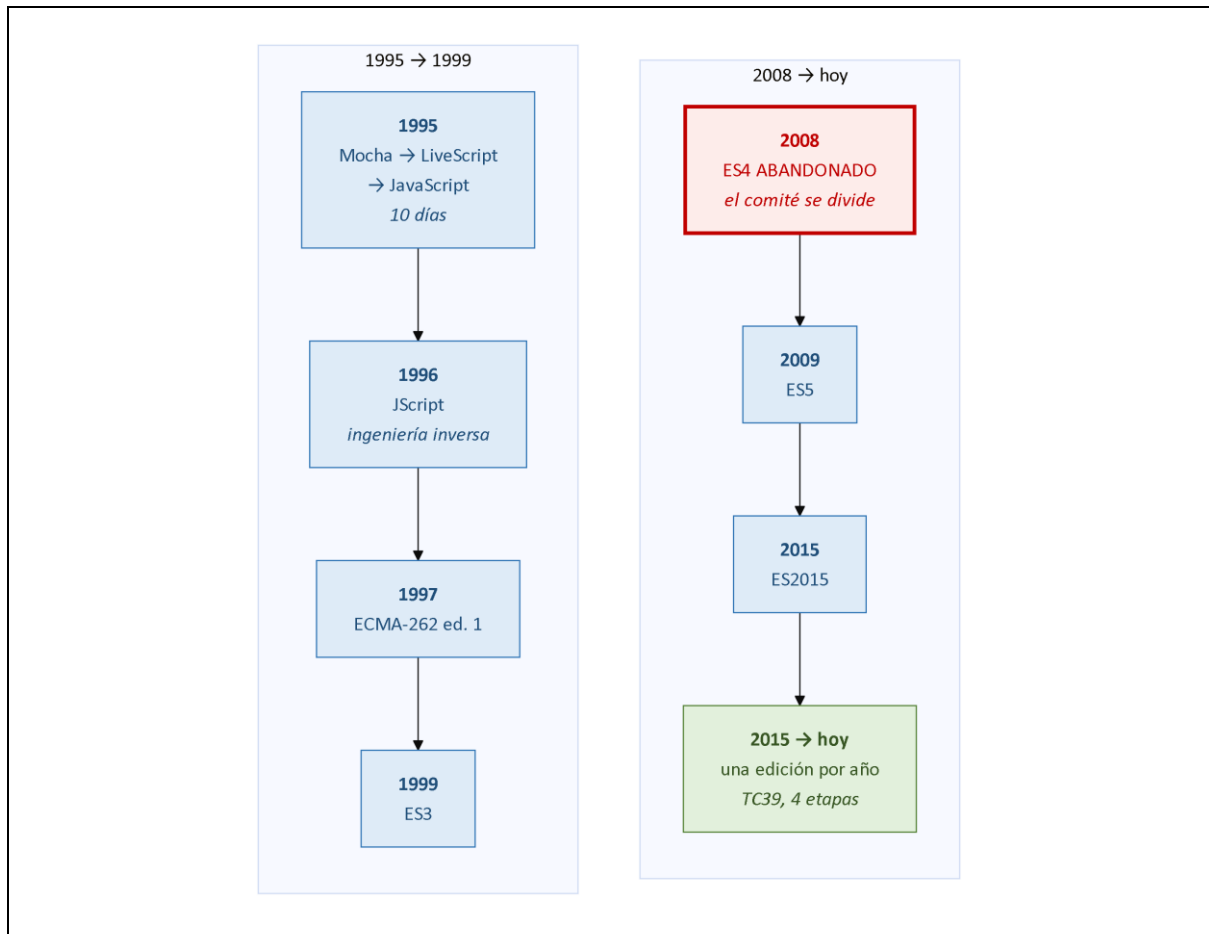


Figura 3.1. De Mocha al ciclo anual

Las cuatro decisiones que explican todo lo demás

Decisión	Qué gana	Qué cuesta
1. No romper la web. Una página de 1997 debe funcionar hoy	Treinta años de código que nunca hubo que reescribir	Ningún error original se puede corregir si hay páginas que dependen de él, y siempre las hay. Todo se agrega al lado
2. Tolerancia antes que rigor. Produce un resultado antes que detenerse	El encargo eran autores no programadores, a quienes un error críptico habría expulsado	Los errores no aparecen donde se cometen sino después, disfrazados (3.5 y 3.12)
3. Objetos dinámicos y prototipos. Sin forma fija ni clases reales, aunque exista <code>class</code>	Flexibilidad enorme, incluso sobre objetos ajenos	Nadie sabe qué forma tiene un objeto que circula: el problema que resuelve TypeScript (3.12)
4. Un solo hilo, coordinado por el bucle de eventos	Ni condiciones de carrera ni bloqueos: nunca vas a necesitar un semáforo	Una operación pesada congela la interfaz entera (3.10)

PARA ENTENDER: los bugs que ya no se pueden arreglar

Casi todo lo que te parece mal diseñado no es un error de diseño. Es un error de diseño que ya no se pudo corregir.

El ejemplo canónico es `typeof null`, que devuelve "object". Es un bug de 1995: los valores se guardaban con una etiqueta de tipo en los bits bajos, la etiqueta `000` significaba «objeto», y `null` era el puntero nulo —todos ceros—, así que se leía como objeto. Lo propusieron arreglar en ES4 y

se rechazó, porque hay código en producción que hace `typeof x === "object"` contando con eso.

Treinta años sosteniendo un bug de una tarde de 1995. Cuando entendés eso, dejás de pelearte con el lenguaje.

3.3 — Cómo y cuándo se ejecuta tu código

Qué dice

El Capítulo 1 mencionó que un script en medio del documento detiene el parseo. Con más precisión: existen **cuatro formas de incluir código**, y la diferencia entre ellas es **cuándo se descarga y cuándo se ejecuta**.

Forma	Descarga	Ejecución	Detiene el parseo
<code><script></code>	Al encontrarlo	Inmediata	Sí, durante ambas
<code><script async></code>	En paralelo	Apenas termina de descargar	Sí, sólo al ejecutar
<code><script defer></code>	En paralelo	Al terminar el parseo, en orden	No
<code><script type="module"></code>	En paralelo	Al terminar el parseo, en orden	No

En criollo

Mirá la primera fila y preguntate lo obvio: **¿por qué el comportamiento por defecto es el peor de los cuatro?** La respuesta está en 1995. Existía `document.write()`, que le permitía a un script **escribir marcado en el punto exacto donde estaba**, y el parser no podía seguir sin saber qué iba a escribir: se detenía, ejecutaba y recién entonces continuaba. Esa función casi no se usa hoy, y **el comportamiento no se puede cambiar igual**, por la primera decisión de diseño de la sección 3.2.

Los **módulos** son la forma que usa el TPI, y traen tres diferencias que no son opcionales: **se cargan diferidos por defecto**, **se ejecutan siempre en modo estricto** (3.12) y **tienen su propio ámbito**, así que una variable declarada en un módulo no contamina el resto de la página. Antes de 2015 todo script compartía un único ámbito global, y dos bibliotecas con el mismo nombre de variable se pisaban en silencio (3.9).

PARA ENTENDER: la diferencia entre ``defer`` y ``async``

Se explica con una sola pregunta: **¿tu script depende de que existan otros?**

- **async ejecuta apenas termina de bajar**, así que con tres scripts el orden es **el orden en que llegaron por la red**: impredecible. Sirve para cosas independientes, como una métrica.
- **defer espera al parseo y respeta el orden en que los escribiste**. Es lo que querés casi siempre.

Y lo práctico: **`type="module"` ya se comporta como defer**. Como el TPI usa módulos no lo vas a sufrir, pero lo vas a ver en código heredado.

3.4 — Tipos: los primitivos y todo lo demás

3.4.1 — Los siete primitivos

Qué dice

JavaScript tiene **siete tipos primitivos y un único tipo compuesto**. Todo lo demás —objetos, arreglos, funciones, fechas— es del tipo `object`.

Tipo	Ejemplo	Notas
<code>number</code>	42, 3.14	Punto flotante de doble precisión. Uno solo
<code>string</code>	"milanesa"	Inmutable
<code>boolean</code>	true	
<code>undefined</code>	undefined	"No se asignó valor"
<code>null</code>	null	"Ausencia deliberada de valor"
<code>symbol</code>	Symbol("id")	Identificador único
<code>bigint</code>	9007199254740993n	Enteros de precisión arbitraria

En criollo

Dos cosas de esa tabla se preguntan siempre.

¿Cuál es la diferencia entre `undefined` y `null`? No es técnica, es de **intención**: `undefined` es lo que pone **el lenguaje** cuando no hay valor; `null`, lo que pone **el programador** para decir que ahí, deliberadamente, no hay valor. En el TPI, un pedido con `descuento: null` es información; `undefined` diría que la propiedad ni vino.

¿Por qué hay un solo tipo numérico? No hay enteros separados de decimales: **todo número es un punto flotante de 64 bits según la norma IEEE 754**, tanto el 2 de una cantidad como el 4750.50 de un precio. Es el punto más caro del capítulo (sección 3.5.4), y de ahí sale RN-F08.

3.4.2 — Valor y referencia: dos nombres para el mismo objeto

Qué dice

Los primitivos se copian **por valor**; los objetos, **por referencia**.

```
let a = 5;
let b = a;
b = 10;
console.log(a);           // 5 — a no se enteró

let p1 = { nombre: "Milanesa", precio: 4750 };
let p2 = p1;
p2.precio = 5000;
console.log(p1.precio); // 5000 — son el mismo objeto
```

En el segundo caso `p1` y `p2` no son dos objetos parecidos: **son dos nombres para el mismo objeto**. La asignación copió la referencia, no el contenido.

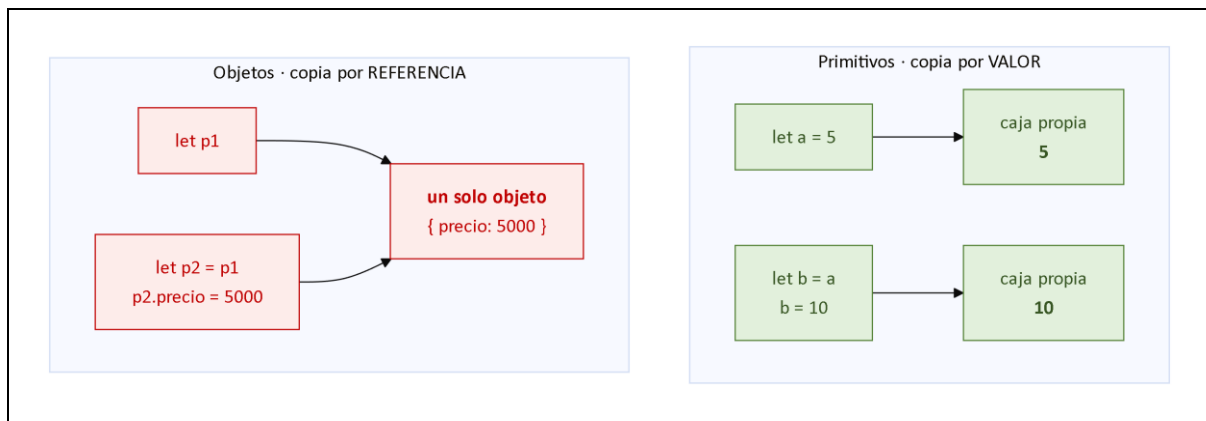


Figura 3.2. Valor y referencia en memoria

En criollo

La analogía que lo fija: **copiar un primitivo es sacarle una fotocopia a un papel; copiar un objeto es hacer una copia de la llave del mismo departamento**. Rayar la fotocopia no toca el original; entrar con la llave sí. Y desde afuera se ven idénticas.

Esto explica que **const no hace inmutable un objeto**: te clava la llave al llavero, no los muebles al piso.

```
const producto = { precio: 4750 };
producto.precio = 5000; // válido: se modifica el objeto
producto = { precio: 5000 }; // TypeError: se reasigna el nombre
```

Para copiar de verdad hay **dos niveles**. El **operador de propagación** copia el primero:

```
const copia = { ...producto };
```

Pero si el objeto contiene otros objetos —y un arreglo es un objeto—, **esos se siguen compartiendo**: adentro del departamento hay una caja fuerte, y de ella también copiaste la llave, no el contenido. Para una copia completa existe `structuredClone()`.

OJO ACÁ: la causa número uno de bugs raros en el frontend

Guardás un producto en un store, lo pasás a una vista, la vista le toca una propiedad para mostrarlo distinto... **y acabás de modificar el original**. Sin error ni advertencia, y lo descubrís tres pantallas después.

Y atrás viene el otro, peor porque creés que ya te cubriste: hacés `{ ...pedido }` convencido de que copiaste, pero `pedido.items` es un arreglo, o sea un objeto, o sea **se sigue compartiendo**.

Cuando en el Capítulo 8 veas **RN-F03** —el estado del servidor vive sólo en el QueryClient y el del cliente sólo en los stores— acordate de esto: **la regla existe para que estas modificaciones cruzadas no puedan ocurrir**.

3.4.3 — El envoltorio que aparece y se va

Los primitivos no tienen propiedades. Ninguna. Y sin embargo esto funciona:

```
"milanesa".toUpperCase();
```

El lenguaje **envuelve temporalmente el primitivo en un objeto**, llama al método y descarta el envoltorio: es el guante descartable. Eso explica que asignarle una propiedad a un primitivo **no falle ni tenga efecto** — se guarda en un envoltorio que se descarta de inmediato. Ni error ni resultado: silencio. La segunda decisión de diseño, otra vez.

3.5 — Coerción: cuando el lenguaje convierte por vos

3.5.1 — Los ocho valores falsos

Qué dice

En un contexto que espera un booleano, **cualquier valor se convierte**. Exactamente **ocho valores se convierten a false**, y todo el resto a true:

false, 0, -0, 0n, "", null, undefined, NaN.

En criollo

Esa lista hay que aprendérsela de memoria, **porque su complemento genera errores que no producen ningún mensaje**. Lo que hay que grabarse es lo que **no** está: **"0" es verdadero** —cadena no vacía—, **[] es verdadero**, **{}** es verdadero y **"false" es verdadero**. La analogía del lenguaje entero: **JavaScript es el traductor que nunca dice «no entiendo»**. Le des lo que le des te devuelve algo, y nunca te avisa si era lo que querías.

OJO ACÁ: el arreglo vacío que siempre entra

Este error lo comete todo el mundo, y en el TPI lo vas a tener servido en bandeja:

```
if (pedido.items) {
  mostrarCarrito();    // se ejecuta SIEMPRE, aunque no haya ni un item
}
```

Un arreglo vacío **es verdadero**, siempre: `[]` no está en la lista de los ocho, y no va a estar nunca, porque cambiarlo rompería la web. Lo correcto es preguntar por lo que te importa:

```
if (pedido.items.length > 0) { ... }
```

Y si `pedido.items` puede ser `undefined`, `pedido.items.length` te tira `TypeError`: ahí usás encadenamiento opcional, `if (pedido.items?.length)`. **Preguntá por el estado real, no por la existencia del contenedor.**

3.5.2 — Las dos igualdades, y cuál usar siempre

=== compara sin convertir: si los tipos difieren, el resultado es `false`. **== convierte antes de comparar**, siguiendo un algoritmo de ECMA-262 que casi nadie recuerda entero, y sus resultados son famosos:

```
"5" == 5           // true
0 == ""           // true
0 == false        // true
null == undefined // true
null == 0         // false
NaN == NaN        // false
```

Mirá las tres últimas líneas juntas: **no hay intuición que sirva**. `null` es igual a `undefined` pero no a `0`, aunque `0` sí sea igual a `""`, y `NaN` no es igual ni a sí mismo. No es una tabla que se deduzca: es una tabla que se consulta.

Usá siempre ===. La única excepción defendible es `== null`, que da verdadero para `null` y `undefined` a la vez: la forma compacta de preguntar por los dos casos de «acá no hay valor» de la sección 3.4.1.

3.5.3 — NaN y los valores que no están

NaN significa «no es un número» y es el resultado de una operación aritmética imposible. Tiene una propiedad que sorprende: **no es igual a sí mismo**, así que se detecta con `Number.isNaN()` — una comparación da siempre `false` y vas a jurar que el valor está bien.

Para valores posiblemente ausentes hay operadores que conviene distinguir:

```
const a = producto.descuento ?? 0;    // usa 0 sólo si es null o undefined
const b = producto.descuento || 0;    // usa 0 también si es 0, "" o false
const calle = pedido?.direccion?.calle;
```

Operador	Cuándo actúa	El error típico		
<code>??</code> · coalescente nulo	Sólo ante <code>null</code> o <code>undefined</code>	Ninguno: es el que casi siempre querés		
<code>**\</code>	<code>\</code>	<code>**</code> · o lógico	Ante cualquiera de los ocho valores falsos	Descarta valores legítimos: un descuento de <code>0</code> , una cantidad de <code>0</code> , una observación vacía
<code>?.</code> · encadenamiento opcional	Accede sólo si el objeto existe ; devuelve <code>undefined</code> en vez de lanzar <code>TypeError</code>	Usarlo de más, tapando un <code>undefined</code> que delataba un bug más arriba		

PARA EL PIZARRÓN: `??` y el o lógico no son intercambiables

El o lógico pregunta «¿esto es falso?». El coalescente nulo pregunta «¿esto no existe?». Confundirlos te hace desaparecer datos válidos.

El caso que te va a pasar: cantidad `|| 1`. Si el usuario puso **cero** a propósito, `0` es falso y tu código lo convierte en uno: acabás de agregarle al pedido un producto que la persona sacó. **Cuando el `0` o la cadena vacía sean datos legítimos —y en pedidos lo son casi siempre—, el operador es `??`.**

3.5.4 — El modelo numérico y el problema del dinero

Qué dice

Acá está el punto más importante del capítulo para el TPI. Todo número de JavaScript es un punto flotante de doble precisión **IEEE 754**. Ese formato representa los números en **base dos**, y hay decimales que en base dos son periódicos, igual que un tercio es periódico en base diez. **`0.1` es uno de ellos.**

```
0.1 + 0.2          // 0.30000000000000004
0.1 + 0.2 === 0.3   // false
```

En criollo

Antes de que empieces a putear al lenguaje: **esto no es un error de JavaScript**. Es el comportamiento de IEEE 754 y ocurre igual en Python, en Java y en C.

La analogía: **es una regla graduada sólo en pulgadas y vos querés medir diez centímetros**. Podés acercarte muchísimo, pero nunca vas a caer sobre una marca, porque no existe ahí; la base dos no tiene una exacta para `0.1`, guarda la más cercana, y al sumar dos aproximaciones los errores se suman. Lo **peligroso** del caso de JavaScript es que **no ofrece alternativa**: Python tiene `Decimal`, Java tiene `BigDecimal`, y acá no hay equivalente. Hay además un segundo límite: **los enteros son exactos sólo hasta $2^{53} - 1$** , `9007199254740991`, disponible como `Number.MAX_SAFE_INTEGER`.

Atá los cabos: **el dinero no tolera ninguno de los dos errores**. Sumar importes en punto flotante acumula centavos que no cierran contra la facturación: en un pedido no se nota, en el cierre de caja del mes sí. Por eso el TPI declara **RN-F08**: todo importe llega como cadena decimal —`"4750.00"`, no `4750`— y se convierte a número **en la capa api/, nunca en la vista**; ninguna aritmética sobre dinero ocurre en el frontend. Del lado del servidor, **la sección 3 del TPI** usa un tipo decimal exacto de la base de datos.

NOTA: por qué el total viene entre comillas

Cuando leas que el total viaja como `"4750.00"` entre comillas, tu primera reacción va a ser pensar que es una complicación al pedo. **Es exactamente al revés**. Si llegara como número, JavaScript lo pasaría a punto flotante apenas lo toque, y cada suma acumularía un error invisible: sin excepción, sin advertencia, sin nada.

La cadena de texto es lo único que garantiza que el número que salió del backend sea idéntico al que se muestra en pantalla.

Y cuando en el Capítulo 6 le pidas a un agente que te tipe la respuesta de la API, fijate bien: **te va a poner total: number sin dudar**. Es lo natural, parece bien, y está mal. Vas a tener que corregirlo vos.

3.6 — Declaraciones, ámbito y clausuras

3.6.1 — `var`, `let` y `const`

Hay tres formas de declarar una variable, y **la primera sólo debería aparecer en código heredado**.

Palabra	Ámbito	¿Se puede reasignar?	¿Se puede redeclarar?
<code>var</code>	Función	Sí	Sí
<code>let</code>	Bloque	Sí	No
<code>const</code>	Bloque	No	No

La diferencia decisiva es el ámbito. **`var` ignora los bloques**: una variable declarada dentro de un `if` o de un `for` existe en toda la función que la contiene, aunque el `if` nunca se haya ejecutado. `let` y `const` respetan el bloque, que es lo que cualquiera espera. **`var` sigue existiendo sólo por la primera decisión de diseño de la sección 3.2, y la práctica actual es usar `const` por defecto y `let` sólo para reasignar**: al leer `const` ya sabés que ese nombre no va a apuntar a otra cosa más abajo.

3.6.2 — Elevación y la zona muerta temporal

Las declaraciones se procesan **antes de que el código se ejecute**: eso es la **elevación**, y se comporta distinto según la palabra usada. Con `var`, la variable existe desde el principio de la función con valor `undefined`:

```
console.log(x); // undefined – existe, sin valor
var x = 5;
```

Con `let` y `const` la variable también existe, pero **acceder a ella antes de su declaración lanza un error**:

```
console.log(y); // ReferenceError
let y = 5;
```

Ese intervalo se llama **zona muerta temporal**. La analogía: **es una caja que ya está en la habitación, rotulada, pero cerrada con candado hasta la línea que la declara**. Con `var` la caja también está, pero abierta y vacía — y meter la mano ahí no da error, da `undefined`.

La zona muerta es **deliberada**: convierte en error visible lo que con `var` era un `undefined` silencioso que se manifestaba cincuenta líneas más abajo. Es uno de los poquísimos casos donde el lenguaje eligió fallar ruidosamente, y **lo pudo hacer sólo porque `let` era nuevo y no rompía nada**.

Las **declaraciones de función** se elevan completas, así que podés invocarlas antes de que aparezcan en el archivo; las asignadas a una variable siguen las reglas de la variable.

OJO ACÁ: el bucle donde `var` muere de verdad

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// imprime: 3, 3, 3
```

¿Por qué tres veces tres? Porque `var i` **es una sola variable** para todo el bucle: las tres funciones capturaron la misma, y para cuando los temporizadores se ejecutan —después, por lo que vas a ver en la sección 3.10— el bucle ya terminó y vale 3.

Cambiá `var` por `let` y salen 0, 1, 2: **`let` crea un enlace nuevo en cada iteración**. No se pudo arreglar `var` —no romper la web—, así que se arregló **creando algo nuevo al lado. Es el patrón de todo el lenguaje**.

3.6.3 — Clausuras: la función que se acuerda

Qué dice

Una **clausura** es una función junto con el entorno léxico donde fue creada. En términos prácticos: **una función recuerda las variables del lugar donde se escribió, aunque se ejecute mucho después y en otro contexto**.

```
function crearContadorDeCarrito() {
  let cantidad = 0; // vive en la clausura
  return {
    agregar() { cantidad += 1; return cantidad; },
    total() { return cantidad; }
  };
}

const carrito = crearContadorDeCarrito();
carrito.agregar(); // 1
carrito.agregar(); // 2
carrito.cantidad; // undefined — no hay forma de tocarla desde afuera
```

En criollo

La analogía es la mochila. **Cuando una función se crea, se lleva puesta una mochila con todas las variables que veía en ese momento** — no copias, las variables mismas. Después se va a otro lado — un temporizador, un manejador, una suscripción— y allá abre la mochila y ahí está todo, vivo.

En el ejemplo, `cantidad` **no existe fuera de la función que la creó** y sigue viva porque las dos funciones devueltas la referencian: así logra JavaScript **estado privado sin clases**. Las clausuras están

en todas partes aunque nadie las nombre: **cada manejador de evento del Capítulo 4, cada función pasada a un `setTimeout`, cada suscripción a un store del Capítulo 8 es una clausura.**

Y tienen una contracara que el Capítulo 4 estudia en detalle: **mientras una clausura viva, todo lo que referencia sigue en memoria.** La mochila no se tira mientras alguien la tenga puesta, así que un manejador que nadie dio de baja mantiene vivo el elemento del DOM al que estaba asociado, aunque ya no esté en la página. **Ese es exactamente el problema que la regla RN-F01 del TPI obliga a resolver.**

3.7 — Funciones, y el `this` que depende de cómo llamás

3.7.1 — Tres formas de escribir una función

```
function calcularTotal(items) { ... }           // declaración: se eleva completa
const calcularTotal = function (items) { ... }; // expresión: sigue a la variable
const calcularTotal = (items) => { ... };       // flecha
```

Las funciones flecha no son sólo una sintaxis más corta, y creer eso es la puerta de entrada a un bug largo: tienen **dos diferencias de comportamiento**, no tienen `this` propio y no tienen el objeto `arguments`. **La primera es la que importa.**

3.7.2 — `this`

Qué dice

En la mayoría de los lenguajes orientados a objetos, `this` es el objeto al que pertenece el método y se sabe leyendo la clase. **En JavaScript `this` depende de cómo se llama la función, no de dónde se escribió.** Las reglas, en orden de prioridad:

Forma de invocación	Valor de <code>this</code>
<code>new Constructor()</code>	El objeto recién creado
<code>fn.call(obj)</code> , <code>fn.apply(obj)</code> , <code>fn.bind(obj)</code>	El objeto indicado
<code>obj.metodo()</code>	<code>obj</code>
<code>fn()</code>	<code>undefined</code> en modo estricto
Función flecha	El <code>this</code> del ámbito donde se escribió

En criollo

Pensalo así: **`this` es como la palabra «acá» en un mensaje de voz.** No significa nada por sí sola: significa el lugar desde donde lo mandás. De ahí sale el error clásico, que en el Capítulo 7 va a reaparecer con los componentes:

```
const carrito = {
  items: [],
  agregar(producto) {
    setTimeout(function () {
      this.items.push(producto); // this es undefined: la función se llamó
    }, 100);
  }
};
```

La función pasada a `setTimeout` **no se invoca como método de nadie: se invoca sola**, cien milisegundos después, desde las entrañas del navegador. La solución es una flecha, que no trae `this` propio y por lo tanto usa el del método que la contiene:

```
setTimeout(() => { this.items.push(producto); }, 100);
```

PARA ENTENDER: la única pregunta que hay que hacerse

No mires dónde está escrita la función. Mirá cómo la están llamando.

¿Hay un punto antes del paréntesis? `carrito.agregar()` → `this` es `carrito`. ¿No hay nada a la izquierda? `agregar()` → `this` es `undefined`.

Y por eso las flechas resuelven el problema: **una flecha no tiene `this` propio**, así que toma el del lugar donde la escribiste. Regla de bolsillo: **para callbacks, flecha siempre**.

3.8 — Objetos, prototipos y clases

Todo objeto tiene un enlace interno a otro objeto, su **prototipo**. Cuando pedís una propiedad que el objeto no tiene, el motor la busca ahí, y si tampoco está, en el prototipo de ese prototipo, hasta llegar a `null`. Eso es **la cadena de prototipos**.

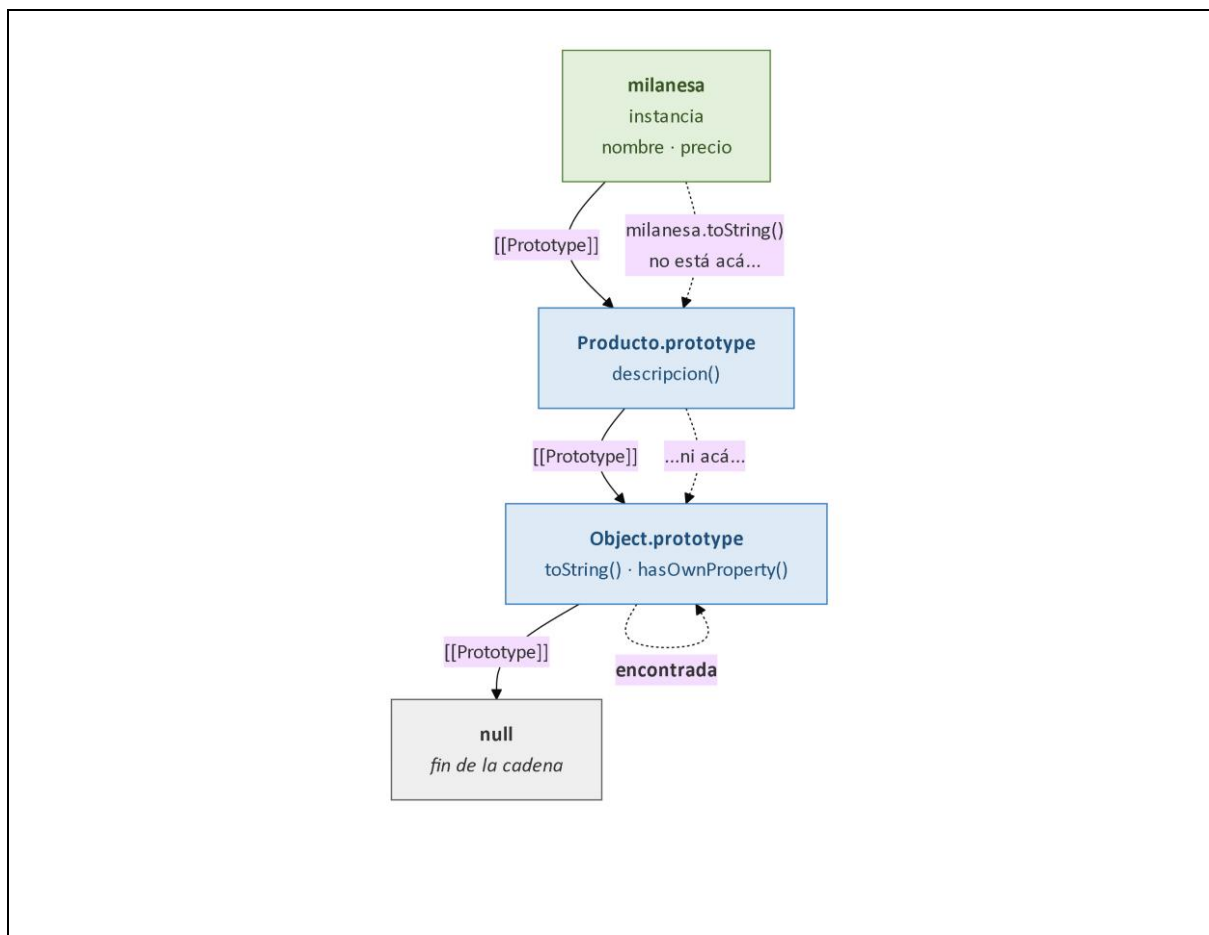


Figura 3.5. La cadena de prototipos

La analogía: **no es un molde, es una cadena de «preguntale al de arriba»** — y esa consulta ocurre **en tiempo de ejecución, cada vez**. Eso explica algo que usás todos los días: **por qué un arreglo tiene `map` sin que nadie se lo haya puesto**. El método vive en `Array.prototype`.

Desde 2015 existe la sintaxis `class`, y es importante entender qué es exactamente:

```
class Producto {
```

```

    constructor(nombre, precio) {
      this.nombre = nombre;
      this.precio = precio;
    }
    descripcion() { return `${this.nombre}`; }
  }

```

class es azúcar sintáctica sobre prototipos. No introduce un modelo nuevo: el método `descripcion` termina en `Producto.prototype`, donde habría quedado escribiéndolo a mano.

Tres comportamientos donde se ve que no es una clase de verdad

El comportamiento	Qué pasa en JavaScript	Qué pasaría en Java o C#
Dónde viven métodos y propiedades	Los métodos, en el prototipo ; las propiedades, en cada instancia. Agregar un método a <code>Producto.prototype</code> afecta a todos, incluso a los ya creados	Imposible: la clase se compiló y los objetos ya se fabricaron
La forma del objeto	No está fija: la clase describe cómo nace el objeto, no cómo sigue	La forma es el contrato: ni un campo más
Qué es la herencia	Delegación, no copia: la instancia recibe un enlace al padre y la búsqueda recorre la cadena cada vez	Copia o tabla de métodos resuelta al compilar

Desde 2022 existen los **campos privados**, que se declaran con almohadilla y son la única forma de privacidad real del modelo de objetos:

```

class Carrito {
  #items = []; // inaccesible desde afuera, de verdad
  agregar(p) { this.#items.push(p); }
  get total() { return this.#items.length; }
}

```

A diferencia de la vieja convención del guion bajo —sólo **un pedido de buena voluntad**—, acceder a `carrito.#items` desde afuera **es un error de sintaxis**: es el equivalente moderno de la privacidad que la sección 3.6.3 lograba con clausuras.

Consecuencia concreta para el Capítulo 7: los componentes web **se declaran obligatoriamente con class** porque el navegador lo exige, pero el objeto resultante sigue siendo dinámico.

PARA ENTENDER: molde contra cadena

Si venís de un lenguaje con clases de verdad, este es el punto donde tenés que cambiar el chip, y conviene hacerlo ahora y no en el Capítulo 7.

En Java o C#, una clase es un molde: definís la forma, se fabrican objetos con esa forma, y la forma no cambia. **En JavaScript, un objeto delega en otro objeto.** No hay molde: hay una cadena, y cuando pedís una propiedad que no está, el motor sube hasta encontrarla o llegar a `null`.

Por eso podés agregarle un método a `Producto.prototype` a las tres de la tarde y **los mil productos de la mañana lo tienen**. `class` te ahorra el andamiaje; **no te cambia el modelo**.

3.9 — Módulos

Antes de 2015 el lenguaje no tenía módulos: todo script compartía el mismo ámbito global, y evitar colisiones dependía de convenciones y de patrones elaborados. Los módulos ES resolvieron eso dentro del propio lenguaje:

```
// api/productos.ts
export async function listarProductos(pagina) { ... }
export const PAGINA_POR_DEFECTO = 1;

// vistas/catalogo.ts
import { listarProductos, PAGINA_POR_DEFECTO } from "../api/productos";
```

Las cuatro propiedades que hay que tener presentes

Propiedad	Qué significa	Para qué sirve
Ámbito propio	Nada se comparte salvo lo que exportes	Se acabaron las colisiones de nombres
import estáticos	Ruta literal, arriba, sin depender de una condición	Deja a las herramientas del Capítulo 7 analizar el grafo sin ejecutar el código y borrar lo que no se usa
Se ejecutan una sola vez	Por más veces que se importen, el cuerpo corre una vez	Hace que un store del Capítulo 8 sea uno solo
Siempre en modo estricto	Sin declararlo y sin poder desactivarlo	Cubre todo el código del TPI (sección 3.12)

Existe también la **importación dinámica**, `import()`, que devuelve una promesa y carga un módulo bajo demanda: es la base de la división del paquete del Capítulo 7.

Dos formas de exportar, y la elección no es indistinta

```
export function listarProductos() { ... } // nombrada
export default function listarProductos() { } // por defecto
```

Forma	Qué permite	El problema
Nombrada	Obliga a usar el mismo nombre : las herramientas detectan un error de tipeo y el editor autocompleta	Ninguno relevante acá
Por defecto	Cada archivo la importa con el nombre que quiera	Esa libertad es el problema : el mismo módulo con tres nombres en tres archivos, y buscar sus usos se vuelve imposible

La arquitectura del TPI, que estudia el Capítulo 8, se apoya en **poder rastrear qué usa qué**: **las exportaciones nombradas son la elección coherente con eso**.

Conviene conocer de antemano las **importaciones circulares**: A importa de B y B importa de A. No dan error inmediato —el sistema de módulos las tolera—, pero uno va a ver al otro **a medio inicializar**, y el síntoma es un `undefined` donde debería haber una función. El diagnóstico es difícil porque **el error aparece lejos de la causa**; la solución casi siempre es extraer lo compartido a un tercer módulo.

NOTA: la restricción que habilita la optimización

Fijate en una propiedad que parece un detalle y es la base de todo el Capítulo 7: **los `import` son estáticos**. No podés hacer `import` adentro de un `if`, y al principio parece una limitación molesta.

Pero eso es exactamente lo que le permite a una herramienta **leer tu código sin ejecutarlo**, armar el grafo completo de dependencias y borrar del paquete final las funciones que no usa nadie.

Esa es la diferencia entre bajar 400 kilobytes de biblioteca o los 12 que realmente usás. **La restricción es la que habilita la optimización** — el mismo intercambio que venimos viendo desde el Capítulo 1.

3.10 — El bucle de eventos

3.10.1 — Un solo hilo

Qué dice

JavaScript en el navegador tiene un único hilo de ejecución, y ese hilo es el mismo que calcula la disposición y pinta la pantalla. No pueden ocurrir dos cosas a la vez: mientras se ejecuta código, no se dibuja; mientras se dibuja, no se ejecuta código.

En criollo

La analogía, y es literal: **es un bar con un solo empleado**. Atiende la caja, prepara los cafés y limpia las mesas. Mientras prepara un café **no hay nadie en la caja**, y la fila que se forma no es una fila lenta: es una fila donde nadie atiende.

La consecuencia es severa. Una función que tarda dos segundos no produce una interfaz lenta: produce una interfaz **muerta** durante dos segundos. Los clics no responden, las animaciones se detienen, el texto no se puede seleccionar, y **el navegador incluso puede ofrecerte cerrar la pestaña**. El usuario del TPI ni siquiera ve el cartel de «cargando» —dibujarlo también requiere el hilo—: ve una aplicación rota.

3.10.2 — Pila, tareas y microtareas

Qué dice

El modelo tiene cuatro piezas:

Pieza	Qué contiene
Pila de llamadas	Las funciones que se están ejecutando ahora
Cola de tareas	Trabajo pendiente: temporizadores, eventos, respuestas de red
Cola de microtareas	Continuaciones de promesas y <code>queueMicrotask</code>
Bucle de eventos	El coordinador que decide qué entra a la pila

El algoritmo del bucle es breve y **hay que saberlo de memoria**:

1. Ejecutar todo lo que haya en la pila hasta vaciarla.
2. **Vaciar por completo la cola de microtareas**. Si una microtarea encola otra, también se ejecuta ahora.
3. Renderizar, si corresponde.
4. Tomar **una sola** tarea de la cola de tareas y volver al paso 1.

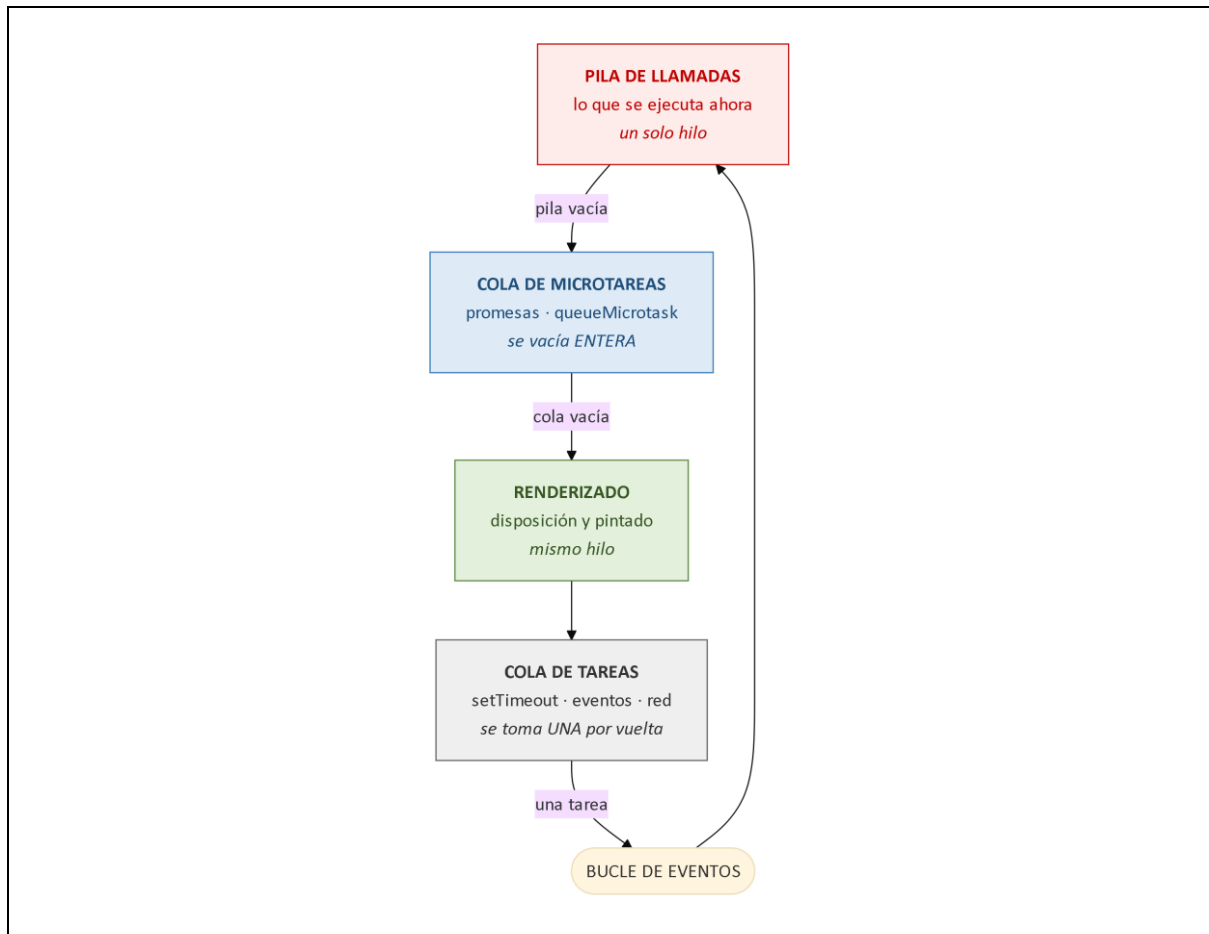


Figura 3.3. Pila, colas y bucle de eventos

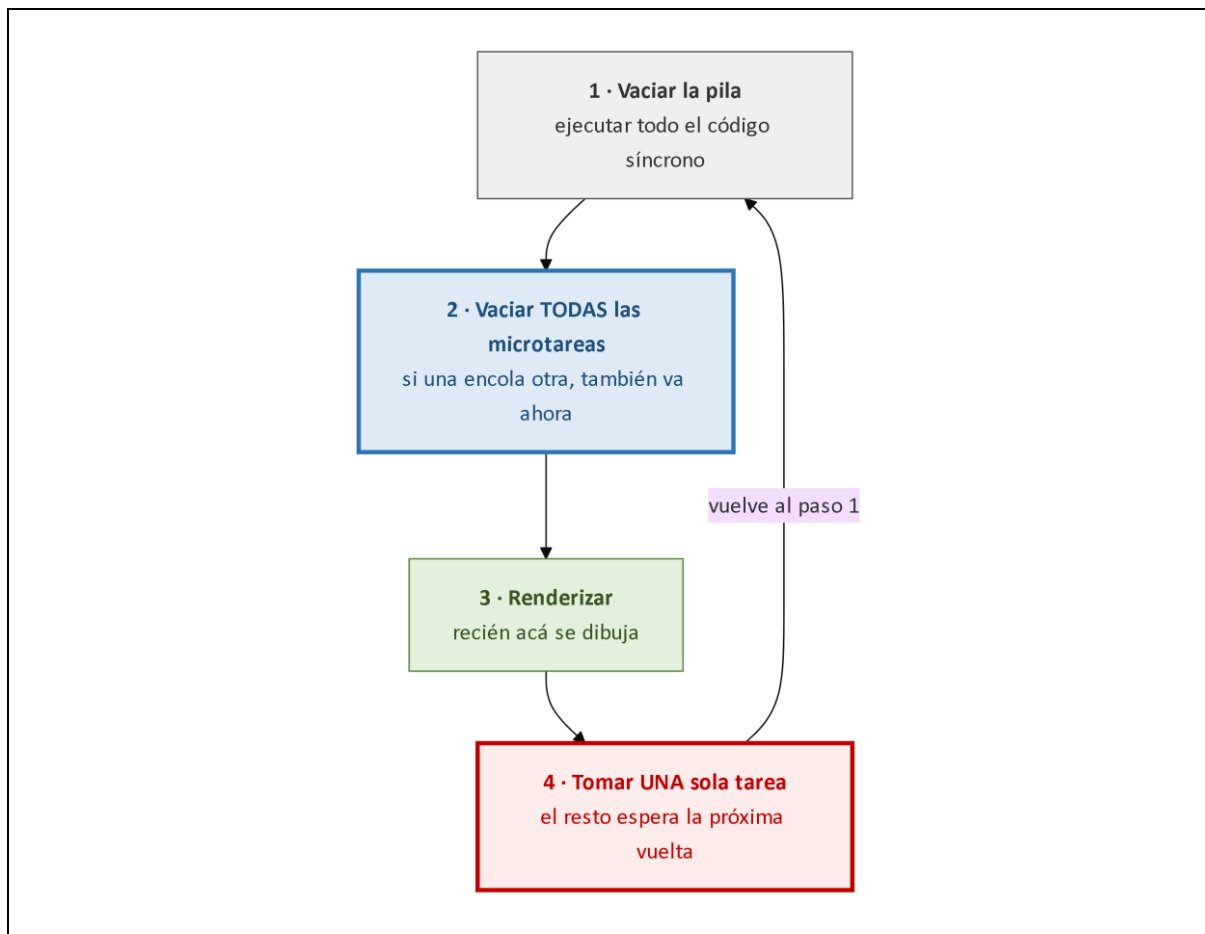


Figura 3.4. El orden de vaciado

En criollo

Volvamos al bar del empleado único, porque cada pieza tiene su equivalente. **La pila de llamadas** es lo que tiene en las manos: si para el café hay que moler granos, apila «moler» sobre «hacer café» y al terminar vuelve a lo de abajo. **La cola de tareas** es la fila de clientes en la puerta: cada temporizador que vence, cada clic, cada respuesta de red. **La cola de microtareas** son los papelitos del mostrador: «cuando termines eso, agregame esto».

El bucle de eventos es la regla que sigue, y las dos asimetrías del paso 2 son todo: **los papelitos se resuelven TODOS antes de llamar al siguiente cliente, y de la fila entra UNO solo por vuelta**. Por eso **una promesa siempre se resuelve antes que un `setTimeout`, incluso uno de cero milisegundos**.

```

console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");

// Imprime: 1, 4, 3, 2
  
```

1 y 4 son código síncrono: lo que el empleado ya tenía en las manos. 3 es una microtarea —el papelito— y sale apenas se vacía la pila. 2 es una tarea —el cliente de la fila— y espera a la vuelta siguiente, aunque su temporizador diga cero. **Ese cero no significa «ya»: significa «poneme en la fila lo antes posible».**

EXPERIMENTO — hacelo hoy, en cualquier página

Poné esto en la consola del navegador, con la página a la vista:

```
const fin = Date.now() + 5000;
while (Date.now() < fin) { /* no hace nada, pero ocupa el hilo */ }
```

Durante esos cinco segundos, probá:

1. **Uno.** Hacer clic en cualquier botón de la página.
2. **Dos.** Seleccionar texto con el mouse.
3. **Tres.** Mirar si alguna animación sigue corriendo.

Nada responde. No está lento: está muerto. Y ojo con el detalle que más impresiona: **los clics que hiciste no se perdieron.** Se encolaron, y cuando el bucle se libera se ejecutan todos juntos de golpe.

Eso es lo que te va a pasar en el TPI si procesás una lista grande de pedidos sin cortar el trabajo: el usuario va a clickear tres veces creyendo que no anduvo, y después **se van a disparar las tres.**

3.10.3 — El mismo problema del otro lado del cable

La sección 1.4 del TPI describe el modelo de ejecución asíncrono del backend, y la sección 5.5 exige que el cálculo del hash de contraseña con bcrypt salga del bucle de eventos mediante un hilo aparte. **Es el mismo problema de esta sección.**

El servidor asíncrono también atiende sobre un bucle de eventos. Una operación que consume procesador sin ceder el control —y bcrypt **está diseñado a propósito para consumir procesador**— bloquea ese bucle, y **ninguna otra petición se atiende**: no sólo se demora quien inicia sesión, se demoran todos. Es el mismo bar con un solo empleado, pero los clientes son mil peticiones HTTP.

NOTA: dos capítulos por el precio de uno

Quien entendió esta sección puede leer las secciones 1.4 y 5.5 del TPI **sin ninguna explicación adicional.** Cambia el lenguaje, cambia la máquina y cambia el lado del cable; **el modelo es idéntico.**

Cuando dos problemas tienen la misma forma, la solución también: **sacar el trabajo pesado del hilo que atiende.** En el navegador, partirlo en fragmentos o mandarlo a un worker; en el servidor, un hilo aparte. Distintas herramientas, **una sola idea.**

3.11 — Herramientas de diagnóstico

A diferencia de CSS, **JavaScript sí falla ruidosamente**, y conviene aprovecharlo. La consola es el primer lugar donde mirar: el mensaje trae el tipo, la descripción y **la traza de llamadas**, que se lee de arriba hacia abajo, donde lo de arriba es lo último que se ejecutó — o sea, dónde explotó, no dónde está el problema.

Las herramientas, y cuándo usar cada una

Herramienta	Qué muestra	Cuándo la usás
<code>console.table()</code>	Un arreglo de objetos como tabla legible	Al mirar la respuesta de un endpoint de listado
<code>console.dir()</code>	Un elemento del DOM como objeto , no como marcado	Cuando querés las propiedades del nodo, no su HTML (Capítulo 4)
<code>console.time()</code> / <code>timeEnd()</code>	Cuánto tardó un fragmento	Medición barata antes del panel de rendimiento

Herramienta	Qué muestra	Cuándo la usás
El depurador	Todo el estado en un momento: cada variable, la cadena de ámbitos —con las clausuras de la sección 3.6.3— y la pila completa	Siempre que el bug no sea evidente
El panel de rendimiento	El hilo principal grabado, con las tareas largas marcadas: las que superan los 50 ms (sección 3.10.2)	Cuando necesitás saber qué función concreta congela la interfaz

Los puntos de interrupción **condicionales** merecen mención aparte: sólo detienen la ejecución cuando se cumple una expresión que escribís vos. Son útiles dentro de un bucle largo, donde parar en cada vuelta es inservible y parar en la del pedido que falla es justo lo que necesitás.

FIGURA 3.6

captura

*Captura del depurador en un punto de interrupción **dentro de una clausura**, con el panel de ámbitos mostrando la variable capturada y la pila de llamadas. Es la evidencia visual de la sección 3.6.3*

Figura 3.6. El depurador detenido

FIGURA 3.7

captura

Captura del panel de rendimiento con una tarea larga marcada en rojo, mostrando el bloqueo del hilo principal descrito en la sección 3.10.1

Figura 3.7. Una tarea larga bloqueando

OJO ACÁ: si depurás con ``console.log``, estás perdiendo tiempo

Un log te muestra **un valor en un momento**, y si no era el que pensabas agregás otro, y otro, y recargás diez veces. **Un punto de interrupción te muestra todo el estado**: todas las variables, toda la cadena de ámbitos, toda la pila de llamadas, y podés avanzar paso a paso.

Hay un detalle que casi nadie usa y vale oro: cuando el depurador está detenido, **la consola ejecuta en ese contexto**. Podés ver el valor de una variable local, o probar una expresión antes de escribirla en el código.

Dedícale veinte minutos: **es la mejor inversión de tiempo de todo el módulo**, y te sirve en cualquier lenguaje.

3.12 — Seguridad y evolución

El **modo estricto**, incorporado en ES5, desactiva comportamientos heredados peligrosos: convierte en error asignar a una variable no declarada —que sin él creaba una global en silencio—, prohíbe duplicar parámetros y hace que `this` sea `undefined` en una función suelta en vez de apuntar al objeto global, que es lo que hace fallar visiblemente el ejemplo de la sección 3.7.2. **Los módulos son estrictos siempre**, así que **todo el TPI lo es**.

Tres riesgos concretos

El riesgo	Cómo se explota	La defensa
Ejecución de texto como código. <code>eval()</code> y el constructor <code>Function</code> ejecutan una cadena como código	Si la cadena trae algo del usuario, el usuario ejecuta código en tu página	No hay uso legítimo acá, y el <code>Content-Security-Policy</code> de la sección 16.5 del TPI los bloquea
Contaminación de prototipos. Todo objeto hereda de <code>Object.prototype</code> : modificarlo afecta a todo el programa	Un JSON con clave <code>__proto__</code> mandado a una función que fusiona objetos sin filtrar	No fusionar datos externos sin validarlos : los esquemas de la sección 7 del TPI
Datos sensibles en el cliente. Lo que llega al navegador es visible y modificable	Abrir las herramientas de desarrollo y cambiar una variable	Base de RN-F04: las guardas de ruta son usabilidad, no seguridad (Capítulo 5)

OJO ACÁ: nada de lo que corre en el navegador es una garantía

Ya lo viste en el Capítulo 1 con el `required` de los formularios, y vuelve acá con el lenguaje entero. El código que escribís se descarga completo en la máquina del usuario: lo puede leer, modificar y saltar, y mandar la petición sin pasar por tu formulario, con el precio que quiera.

Por eso **RN-F04 dice que las guardas de ruta son usabilidad**: sirven para que alguien no entre por error donde no le corresponde, no para frenar a alguien decidido a entrar. Eso lo tiene que impedir el servidor, en cada endpoint, siempre.

Si alguna vez pensás «esto ya lo controlé en el frontend», **pará: acabás de abrir un agujero**.

Y de acá sale TypeScript, que es el próximo problema

Todo lo que este capítulo describió —la coerción, el `this` variable, los objetos sin forma fija, los errores que no aparecen hasta ejecutar— **es manejable en un archivo de doscientas líneas, y a escala de un sistema como el TPI deja de serlo**: no por falta de disciplina, sino porque **nadie recuerda qué forma tiene cada objeto que circula por el programa**. De ahí sale TypeScript, y por eso el módulo lo estudia después: **para entender qué problema resuelve hay que haber tenido el problema**.

3.13 — Verificación: el checklist honesto

Nueve comprobaciones. **No son ejercicios: son el criterio para saber si el capítulo se entendió.**

- Predecir el orden de salida de un fragmento con código síncrono, `setTimeout` y promesas, y **verificarlo en la consola**. (3.10.2)
- Explicar por qué modificar una propiedad de un objeto `const` es válido y reasignarlo no. (3.4.2)
- Copiar un objeto anidado con el operador de propagación, modificar el nivel interno de la copia y **verificar que el original cambió**. (3.4.2)
- Escribir una función que devuelva un contador con estado privado mediante una clausura, y comprobar en el depurador que la variable **no es accesible desde afuera**. (3.6.3)
- Provocar deliberadamente un error de `this` con una función común dentro de un método, y corregirlo con una flecha. (3.7.2)
- Enumerar los ocho valores falsos y **verificar cada uno en la consola**. (3.5.1)
- Evaluar `0.1 + 0.2` y explicar **por qué** el resultado no es `0.3`, y qué estrategia adopta el TPI ante ese límite. (3.5.4)
- Detener la ejecución en un punto de interrupción dentro de una clausura y **ubicar la variable capturada en el panel de ámbitos**. (3.11)
- Bloquear el hilo principal con un bucle y **documentar qué deja de funcionar** en la página durante ese lapso. (3.10.1)

3.14 — Los diez errores frecuentes

Todos tienen algo en común: **en el momento, no parecen errores**. Por eso son frecuentes.

El error	Por qué duele	Sección
Comparar con <code>==</code> en lugar de <code>===</code>	Produce igualdades que nadie esperaba, como <code>0 == ""</code> , y ese <code>true</code> se ve igual que uno correcto	3.5.2
Creer que <code>const</code> congela un objeto	Impide reasignar el nombre, no modificar el contenido : un dato que creías protegido aparece cambiado	3.4.2
Copiar en superficie creyendo que se copió todo	La propagación copia un nivel ; los anidados se siguen compartiendo. Peor que no copiar, porque da sensación de cobertura	3.4.2
Usar el <code>o</code> lógico para valores por defecto	Actúa ante cualquier valor falso, así que descarta un <code>0</code> o una cadena vacía legítimos . Para eso está <code>??</code>	3.5.3
Comparar contra <code>NaN</code> con <code>===</code>	<code>NaN</code> no es igual a sí mismo : da siempre <code>false</code> y el valor inválido pasa de largo	3.5.3
Operar con dinero en punto flotante	Acumula centavos que no cierran contra la facturación . Es lo que RN-F08 previene	3.5.4
Perder <code>this</code> en un callback	Una función común pasada a un temporizador no conserva el <code>this</code> del método	3.7.2

El error	Por qué duele	Sección
Suponer que <code>setTimeout(fn, 0)</code> ejecuta ya	Encola una tarea, y las microtareas pendientes se ejecutan antes	3.10.2
Bloquear el hilo con un cálculo largo	La interfaz no se pone lenta: deja de responder	3.10.1
Depurar sólo con <code>console.log</code>	Muestra un valor en un momento; el depurador, todo el estado	3.11

3.15 — Las actividades, y qué busca cada una

Siete actividades. Debajo de cada una, lo que en realidad quiere que descubras.

1. Predicción del bucle de eventos

Dado un fragmento con cinco operaciones —código síncrono, dos `setTimeout` y dos promesas encadenadas—, escribir el orden de salida **antes** de ejecutarlo, ejecutarlo y explicar cada diferencia.

Qué busca: *que la predicción falle al menos una vez: ahí el algoritmo de la sección 3.10.2 deja de ser una lista y pasa a ser algo que sabés.*

2. Referencias en el dominio del TPI

Escribir una función que reciba un pedido con sus ítems y devuelva una versión con descuento aplicado, **sin modificar el original**. Verificar con el depurador que el pedido de entrada quedó intacto y **documentar qué falló en el primer intento**.

Qué busca: *que te comas el bug de la copia superficial con las manos en la masa: pide el primer intento porque se da por descontado que falla.*

3. Auditoría de coerción

Para una lista dada de quince comparaciones con `==`, predecir el resultado de cada una, verificarlo y **explicar la conversión que aplicó el motor** donde la predicción falló.

Qué busca: *que compruebes que no hay intuición posible, y que de ahí salga por convicción propia la regla de usar siempre `===`.*

4. El problema del dinero

Sumar cien veces `0.1` en un bucle, comparar con `10` y calcular la diferencia acumulada. Repetir la suma **trabajando en centavos con enteros** y comparar. Relacionar con RN-F08.

Qué busca: *ver la diferencia crecer: un error de una diezmilésima no impresiona a nadie, pero cien sumas después la cifra convence sola.*

5. Estado privado con clausuras

Implementar un carrito con `agregar`, `quitar` y `total`, cuyo estado interno **no sea accesible desde afuera**. Demostrar en la consola que no hay forma de modificarlo.

Qué busca: *que uses una clausura para un problema real y no de laboratorio: es el mismo patrón de los stores del Capítulo 8.*

6. Exploración: el costo de bloquear

Procesar un arreglo de cincuenta mil elementos de forma síncrona y medir con el panel de rendimiento cuánto bloquea el hilo. Reescribirlo partiendo el trabajo en fragmentos que cedan el control, comparar ambas grabaciones y relacionarlo con la sección 3.10.1. *(Requiere el panel de rendimiento del navegador.)*

Qué busca: *que veas la tarea larga marcada en rojo con tus propios datos, y que la segunda grabación muestre que el trabajo total no bajó: lo que cambió es que ahora hay huecos donde el navegador puede dibujar.*

7. Exploración: la compatibilidad hacia atrás

Buscar en la especificación de ECMAScript **tres comportamientos documentados como heredados o desaconsejados** que no se pueden eliminar. Para cada uno, explicar qué rompería su eliminación y relacionarlo con la primera decisión de diseño de la sección 3.2. *(Requiere consultar tc39.es/ecma262/.)*

Qué busca: *que abras la norma y veas que el comité documenta sus propias cicatrices. Después de eso, la idea madre del capítulo deja de ser una frase que te dijeron.*

3.16 — Síntesis: las once frases

- JavaScript se diseñó en diez días como **lenguaje de pegamento para no programadores**, con la restricción de marketing de parecerse a Java. **Parece Java y funciona como Scheme**: esa distancia explica la mayoría de las confusiones.
- La decisión rectora es **no romper la web**. Ningún error del diseño original se corrigió jamás; todo se agregó al lado. `typeof null === "object"` es un bug de 1995 que sigue vigente porque arreglarlo rompería páginas existentes.
- El lenguaje **prefiere producir un resultado antes que detenerse**. Es tolerancia deliberada, **la misma decisión que el HTML y el CSS toman en sus propios terrenos**.
- Los primitivos se copian **por valor** y los objetos **por referencia**. `const` impide reasignar el nombre, no modificar el contenido, y la propagación copia **un solo nivel**.
- 0.1 + 0.2 no es 0.3**: no es culpa de JavaScript sino de IEEE 754, pero a diferencia de otros lenguajes **acá no hay tipo decimal exacto**. De ese límite sale RN-F08: el dinero viaja como cadena y no se opera en el frontend.
- this depende de cómo se invoca la función, no de dónde se escribió**. Las funciones flecha no tienen `this` propio y por eso son la elección correcta para callbacks.
- Una **clausura** es una función más el entorno donde nació: da estado privado sin clases, y **mantiene vivo en memoria todo lo que referencia** — el origen del problema que RN-F01 obliga a resolver.
- class es azúcar sintáctica sobre prototipos**. No introduce un modelo de objetos nuevo: hay una cadena de delegación, no un molde.
- El **bucle de eventos** vacía las microtareas **por completo** y toma las tareas **de a una**. Por eso una promesa siempre se resuelve antes que un `setTimeout(0)`.
- Un solo hilo ejecuta y dibuja**. Bloquearlo no da una interfaz lenta sino una muerte: el mismo problema que el TPI enfrenta del lado del servidor en sus secciones 1.4 y 5.5.
- Todo lo anterior es manejable en doscientas líneas y **deja de serlo a escala de sistema**: ese es el problema que TypeScript resuelve, y por eso se estudia después. **Para entender la solución hay que haber tenido el problema**.

3.17 — Qué leer, y en qué orden

El original lista las fuentes en dos párrafos densos. Acá van ordenadas por prioridad real.

Si lees una sola cosa

Simpson, *You Don't Know JS Yet* (2.ª edición, de lectura libre en su repositorio público). Trata **con mayor profundidad los tres temas difíciles de este capítulo** —tipos y coerción, ámbito y clausuras, y `this` con prototipos—, con un volumen para cada uno. Si lees un solo tomo, que sea el de ámbito y clausuras.

Si lees tres

- **Roberts**, *What the heck is the event loop anyway?* (JSConf EU, 2014): la **explicación visual más clara** del modelo de la sección 3.10, en veintiséis minutos.
- **Archibald**, *In the Loop* (JSConf Asia, 2018): la diferencia entre tareas y microtareas **con el detalle que esa distinción requiere**.
- **Wirfs-Brock y Eich**, *JavaScript: The First 20 Years* (ACM HOPL IV, 2020): el relato más completo del contexto histórico de la sección 3.2, **escrito por quienes participaron del proceso**.

Y para consulta cotidiana: la documentación de **MDN** en `developer.mozilla.org`, **la más confiable y la que conviene adoptar como fuente por defecto**, por encima de cualquier resultado de buscador.

Las fuentes normativas (para consultar, no para leer de corrido)

- **El lenguaje**: la especificación **ECMA-262**, del comité TC39, publicada como **estándar viviente** en `tc39.es/ecma262`. Pertinentes acá: **tipos y valores**, que define los siete primitivos de la sección 3.4; **operaciones abstractas de conversión**, que documenta el algoritmo de coerción de la sección 3.5; y **tareas y trabajos**, que formaliza la cola de microtareas de la sección 3.10.
- **El bucle de eventos: no está en ECMA-262 sino en el estándar HTML del WHATWG**, en su modelo de procesamiento de eventos. La distinción importa: **el bucle es del entorno de ejecución, no del lenguaje**, y por eso el mismo lenguaje corre con otro bucle del lado del servidor (sección 3.10.3).
- **El proceso del comité**: las cuatro etapas de una propuesta, en `tc39.es/process-document`; el repositorio público muestra el estado de cada una.
- **El formato numérico**: la norma **IEEE 754-2019**, de donde sale todo lo de la sección 3.5.4. Su lectura no es necesaria, pero **sí saber que existe y que el problema no es propio de JavaScript**.

Cierre: las siete cosas que hay que recordar

Si dentro de un mes te acordás de siete frases de todo esto, que sean estas.

LAS SIETE

1. El lenguaje **no se pudo corregir nunca**: casi todo lo que parece mal diseñado es un error de 1995 con un arreglo puesto al lado.
2. **Parece Java y funciona como Scheme**. Leerlo esperando lo primero es equivocarse en todo lo que importa.
3. **Los objetos se copian por llave, no por fotocopia**. Y el operador de propagación copia un solo piso del edificio.
4. **`0.1 + 0.2` no es `0.3`**. De ahí sale que el dinero viaje como cadena de texto — no del capricho del enunciado.
5. **`this` no se lee: se pregunta cómo llamaron a la función**. ¿Hay un punto a la izquierda? Ese es el `this`. Para callbacks, flecha siempre.
6. **Una clausura es una mochila**. Da estado privado, y mantiene vivo todo lo que hay adentro — incluido un nodo del DOM que ya nadie ve.

7. Hay un solo empleado en el bar. Los papelitos del mostrador se resuelven todos; de la fila entra uno por vez. Y mientras el empleado trabaja, **nadie atiende la caja.**

Y una octava, que no está escrita en el capítulo pero está en todas sus páginas: **cuando algo de este lenguaje parece absurdo, busca la restricción que lo originó.** Casi siempre es «no romper la web», y casi siempre la versión vieja sigue ahí porque alguien todavía la usa. Eso es lo que separa aprender JavaScript de padecerlo.

Continúa en: Capítulo 4 — El DOM: programar la página sin framework, donde el lenguaje de este capítulo se aplica sobre el árbol de nodos del Capítulo 1, y donde las clausuras de la sección 3.6.3 revelan su costo en memoria.